Codility Practice: Five Problems

Name: [Your Name]
GitHub repository: [https://github.com/your-username/codility-solutions]
AI conversation: [link to this AI conversation]

1. FrogJmp

Problem: https://app.codility.com/programmers/lessons/3-time_complexity/frog_jmp/

1. Handwritten pseudocode

[Add pseudocode/frog_jmp.jpg]

2. Python code

```
"""
Codility Lesson 3 (Time Complexity) - FrogJmp
https://app.codility.com/programmers/lessons/3-time_complexity/frog_jmp/

A frog starts at position X and wants to reach a position >= Y.
Each jump covers exactly D units. Return the minimum number of jumps.

Time: O(1)   Space: O(1)
"""


def solution(X, Y, D):
    # Distance the frog still has to cover
    distance = Y - X

    # We need ceil(distance / D) jumps.
    # (distance + D - 1) // D is integer ceiling division,
    # so we avoid floating-point rounding errors.
    return (distance + D - 1) // D


if __name__ == "__main__":
    print(solution(10, 85, 30))  # expected 3
```

3. Citations

Problem statement: Codility Lessons,
https://app.codility.com/programmers/lessons/3-time_complexity/frog_jmp/
AI tool: Claude (Anthropic), used to help draft the solution and pseudocode.

4. Links

GitHub: [https://github.com/your-username/codility-solutions]
AI conversation: [link to this AI conversation]

2. PermCheck

Problem: https://app.codility.com/programmers/lessons/4-counting_elements/perm_check/

1. Handwritten pseudocode

[Add pseudocode/perm_check.jpg]

2. Python code

```
"""
Codility Lesson 4 (Counting Elements) - PermCheck
https://app.codility.com/programmers/lessons/4-counting_elements/perm_check/

Return 1 if array A (length N) is a permutation of 1..N, otherwise 0.

Time: O(N)   Space: O(N)
"""


def solution(A):
    n = len(A)
    # seen[v] becomes True once value v has appeared (index 0 is unused)
    seen = [False] * (n + 1)

    for value in A:
        # A value outside 1..N can never be part of a permutation of 1..N
        if value < 1 or value > n:
            return 0
        # A repeated value means some other value in 1..N must be missing
        if seen[value]:
            return 0
        seen[value] = True

    # N values, all in range 1..N and all distinct -> every value appears once
    return 1


if __name__ == "__main__":
    print(solution([4, 1, 3, 2]))  # expected 1
    print(solution([4, 1, 3]))     # expected 0
```

3. Citations

Problem statement: Codility Lessons,
https://app.codility.com/programmers/lessons/4-counting_elements/perm_check/
AI tool: Claude (Anthropic), used to help draft the solution and pseudocode.

4. Links

GitHub: [https://github.com/your-username/codility-solutions]
AI conversation: [link to this AI conversation]

3. MissingInteger

Problem: https://app.codility.com/programmers/lessons/4-counting_elements/missing_integer/

1. Handwritten pseudocode

[Add pseudocode/missing_integer.jpg]

2. Python code

```
"""
Codility Lesson 4 (Counting Elements) - MissingInteger
https://app.codility.com/programmers/lessons/4-counting_elements/missing_integer/

Return the smallest positive integer (greater than 0) that does not occur in A.

Time: O(N)   Space: O(N)
"""


def solution(A):
    n = len(A)
    # The answer is always in the range 1..N+1:
    # with only N numbers, at least one of 1..N+1 must be missing.
    # present[v] is True if v (1 <= v <= N+1) appears in A.
    present = [False] * (n + 2)

    for value in A:
        # Ignore non-positive numbers and numbers too big to matter
        if 1 <= value <= n + 1:
            present[value] = True

    # The first value not marked as present is the answer
    for candidate in range(1, n + 2):
        if not present[candidate]:
            return candidate

    return n + 2  # never reached, kept as a safe fallback


if __name__ == "__main__":
    print(solution([1, 3, 6, 4, 1, 2]))  # expected 5
    print(solution([1, 2, 3]))           # expected 4
    print(solution([-1, -3]))            # expected 1
```

3. Citations

Problem statement: Codility Lessons,
https://app.codility.com/programmers/lessons/4-counting_elements/missing_integer/
AI tool: Claude (Anthropic), used to help draft the solution and pseudocode.

4. Links

GitHub: [https://github.com/your-username/codility-solutions]
AI conversation: [link to this AI conversation]

4. MaxCounters

Problem: https://app.codility.com/programmers/lessons/4-counting_elements/max_counters/

1. Handwritten pseudocode

[Add pseudocode/max_counters.jpg]

2. Python code

```
"""
Codility Lesson 4 (Counting Elements) - MaxCounters
https://app.codility.com/programmers/lessons/4-counting_elements/max_counters/

There are N counters, all starting at 0. For each operation A[K]:
  - if 1 <= A[K] <= N : increase counter A[K] by 1
  - if A[K] == N + 1  : set every counter to the current maximum value
Return the final values of the counters.

Setting all N counters on every "max counter" operation would be O(N * M).
Instead we apply it lazily: remember the value every counter should have at
least (base) and only lift a counter up to that value when it is touched.

Time: O(N + M)   Space: O(N)
"""


def solution(N, A):
    counters = [0] * N
    base = 0         # value all counters were raised to by the last "max counter"
    current_max = 0  # highest counter value seen so far

    for op in A:
        if 1 <= op <= N:
            i = op - 1  # convert the 1-based counter number to a 0-based index
            # Apply any pending "max counter" to this counter before increasing it
            if counters[i] < base:
                counters[i] = base
            counters[i] += 1
            # Keep track of the largest counter value
            if counters[i] > current_max:
                current_max = counters[i]
        else:
            # op == N + 1: do not loop over all counters, just record the new base
            base = current_max

    # Counters never touched after the last "max counter" still need the base
    for i in range(N):
        if counters[i] < base:
            counters[i] = base

    return counters


if __name__ == "__main__":
    print(solution(5, [3, 4, 4, 6, 1, 4, 4]))  # expected [3, 2, 2, 4, 2]
```

3. Citations

Problem statement: Codility Lessons,
https://app.codility.com/programmers/lessons/4-counting_elements/max_counters/
AI tool: Claude (Anthropic), used to help draft the solution and pseudocode.

4. Links

GitHub: [https://github.com/your-username/codility-solutions]
AI conversation: [link to this AI conversation]

5. PassingCars

Problem: https://app.codility.com/programmers/lessons/5-prefix_sums/passing_cars/

1. Handwritten pseudocode

[Add pseudocode/passing_cars.jpg]

2. Python code

```
"""
Codility Lesson 5 (Prefix Sums) - PassingCars
https://app.codility.com/programmers/lessons/5-prefix_sums/passing_cars/

A[i] == 0 means a car travelling east, A[i] == 1 a car travelling west.
A pair (P, Q) with 0 <= P < Q < N is passing when A[P] == 0 and A[Q] == 1.
Return the number of passing pairs, or -1 if it exceeds 1,000,000,000.

Time: O(N)   Space: O(1)
"""

LIMIT = 1_000_000_000


def solution(A):
    east_count = 0  # number of east-bound cars seen so far (to the left)
    passing = 0     # number of passing pairs found

    for car in A:
        if car == 0:
            # An east-bound car will pass every west-bound car to its right
            east_count += 1
        else:
            # A west-bound car is passed by every east-bound car to its left
            passing += east_count
            # Stop early once the result is too large
            if passing > LIMIT:
                return -1

    return passing


if __name__ == "__main__":
    print(solution([0, 1, 0, 1, 1]))  # expected 5
```

3. Citations

Problem statement: Codility Lessons,
https://app.codility.com/programmers/lessons/5-prefix_sums/passing_cars/
AI tool: Claude (Anthropic), used to help draft the solution and pseudocode.

4. Links

GitHub: [https://github.com/your-username/codility-solutions]
AI conversation: [link to this AI conversation]